Built Fast, Broken Easy

A Study on the Cybersecurity Risks Emerging from AI Assisted Application Development in Hackathon and Student Building Culture

Abhirup Payne

Independent Researcher, Application and Web Security

September 2026

Abstract

Artificial intelligence coding assistants have removed the traditional entry barrier to software development. A person with no formal programming background can now describe an idea in plain language and, within hours, produce a working web application complete with a login system, a database, and a connection to a third party service. This shift, informally known among student developers as vibe coding, has fuelled a wave of participation in hackathons, college projects, and personal builds. This paper examines the security consequences of that shift. It argues that while AI tools have made the act of building dramatically easier, they have not made the act of securing equally easy, and in many cases they hide the very decisions a builder would need to understand in order to stay safe. The paper reviews the categories of weakness most commonly introduced into AI generated applications, examines the particular danger of exposed API credentials, discusses how hackathon culture quietly pushes prototypes into long term public use, and considers how the habit of publicly showcasing projects on platforms such as LinkedIn can unintentionally hand attackers a map of a system's weaknesses. The paper closes with a practical, non technical checklist that student developers can apply before they deploy or publish a project.

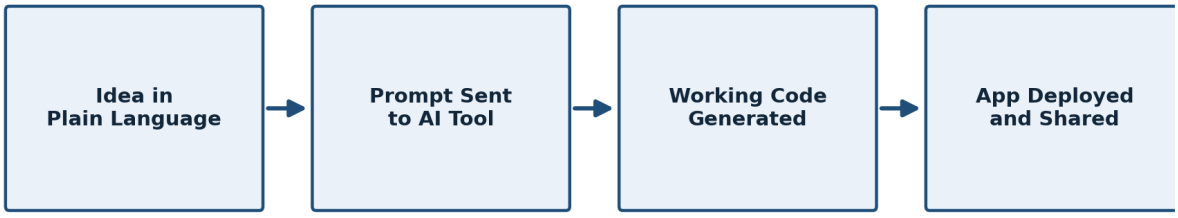
Keywords: vibe coding, application security, API security, hackathon culture, AI assisted development, authentication, authorization, credential exposure

1. Introduction

There has been a quiet but significant change in who gets to build software. For most of the history of computing, building a working application required years of accumulated knowledge: how a server talks to a browser, how a database stores and retrieves records, how a session proves that a user is who they say they are. Today, a large part of that knowledge can be substituted with a well written prompt. Tools built on large language models can generate a login page, wire up a payment form, or connect an application to an external service in a matter of minutes, and the person directing this process does not need to read a single line of the resulting code to see it work on their screen.

This is, on the whole, a genuinely positive development. It has opened participation in hackathons, Google Developer Group events, college capstone projects, and personal side projects to people who would previously have been excluded simply because they did not know how to code. The purpose of this paper is not to argue against that trend. The purpose is to describe, carefully and without alarmism, a side effect of that trend that receives far less attention than it deserves: applications are being built by people who understand what the application does but not always how it works underneath, and that gap is exactly where security weaknesses tend to live.

Figure 1: The Vibe Coding Workflow



Speed increases at every step, but no step independently checks whether the result is secure.

Figure 1. The vibe coding workflow moves from idea to live application in a handful of steps, but no step in this chain independently checks whether the result is secure.

2. The Rise of Vibe Coding and the Democratization of Development

The term vibe coding describes a style of development where a person converses with an AI assistant, describes what they want in natural language, and accepts the generated code largely on faith because it produces the desired visible result. It is a natural extension of low code and no code movements, but it goes further, because the AI assistant is capable of writing genuinely novel, full stack code rather than assembling pre built blocks.

This shift shows up clearly in a few concrete ways:

- A student with no programming background can produce a working prototype, including a login system and a database, within a single afternoon.
- Hackathons increasingly reward ambitious ideas over raw coding ability, because AI assistance closes much of that gap on its own.
- The volume of publicly deployed student and hobby projects has grown sharply, since the cost of trying an idea has dropped close to zero.
- The average depth of understanding a builder has about their own application has gone down even as the speed of building has gone up.

A developer who hand wrote every line of an authentication system, however imperfectly, was forced to at least decide how passwords would be checked and how sessions would be issued. A developer who asked an assistant to add login functionality receives a decision that has already been made for them, often without being told what that decision was or what it assumes about the rest of the system.

3. Why Speed Without Understanding Creates Risk

Security is rarely broken by a single dramatic flaw. It is broken by an accumulation of small, reasonable seeming choices, each of which made sense in isolation to whoever made it. An AI assistant generating code one request at a time is especially prone to producing this kind of accumulation, because it optimizes for satisfying the immediate instruction rather than for reasoning about the security posture of the finished system.

Consider a simple, realistic example:

- A builder asks for a way to fetch a user's order history by their account number.
- The assistant produces code that does exactly that, without independently verifying who is allowed to request which account number.
- The feature works perfectly for the builder testing their own account.
- The same feature also allows any logged in user to read any other user's order history simply by changing a number in the request.

This is not a flaw unique to AI generated code. It is a flaw that has existed in software for as long as software has existed. What has changed is the volume of people now capable of introducing it, and the speed at which they can introduce it, without ever being exposed to the vocabulary needed to recognize it. A builder cannot search for a problem they do not know has a name.

4. Common Weaknesses Found in AI Generated Applications

Reviewing a broad cross section of hackathon and student projects reveals the same handful of weaknesses appearing again and again, almost regardless of the specific idea being built. Each is simple to explain once named, even to someone with no security background.

Authentication that does not reach every sensitive function. Many generated applications correctly hide sensitive pages from the navigation menu but forget to protect the underlying functions those pages call, leaving admin actions reachable by anyone who guesses the right web address.

Authorization that does not distinguish between users. A system can have flawless login checks and still fail here completely. The most common real world version of this is a request containing an identifier, such as an order or profile number, where the server trusts whatever number is sent instead of confirming it belongs to the requester.

APIs that trust the frontend too much. Many builders treat the visual interface as the security boundary of their application. In reality, anyone can bypass the interface entirely and send requests directly to the server. Every check that matters must exist on the server, not only in the browser.

Databases that expose more than intended. Several popular backend platforms used in fast prototyping ship with permissive default settings so a new project works immediately. Builders under time pressure frequently leave these defaults in place.

Secrets stored where anyone can find them. API keys, database passwords, and access tokens are sometimes placed directly inside browser facing code or committed to a public repository, where automated tools can find them within minutes.

Files that were never meant to be public. Configuration files, environment files, backups, and logs occasionally get deployed alongside the intended application, each capable of meaningfully assisting an attacker.

Business logic that assumes good faith. Many applications are built around the assumption that users will follow the intended path through the product. An attacker has no obligation to follow that path and will skip, repeat, or reorder steps to see what the system allows.

Figure 2: Why the Frontend Is Not a Security Boundary

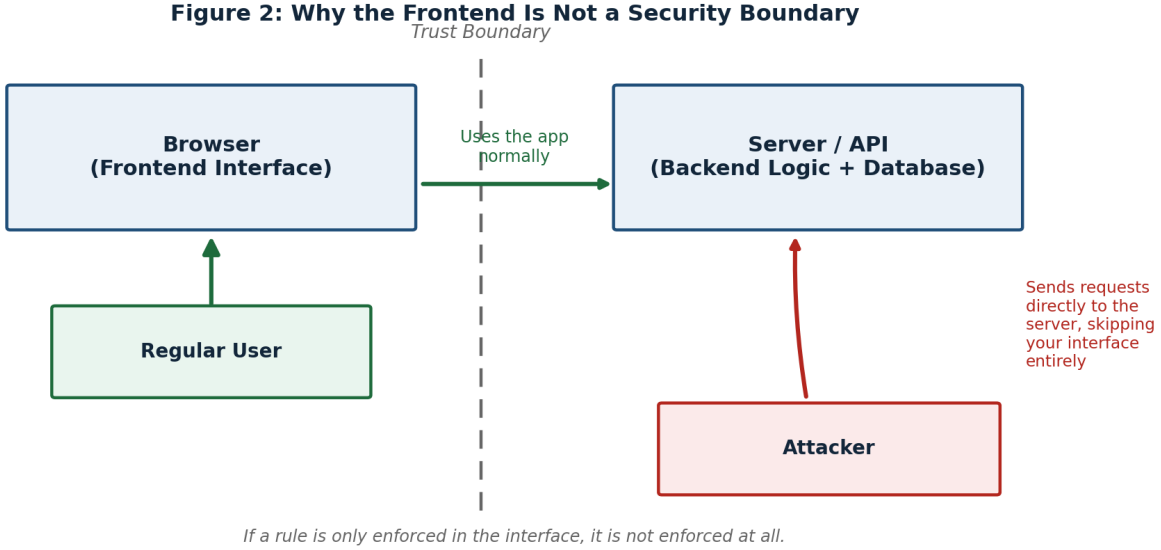


Figure 2. The browser and the server sit on opposite sides of a trust boundary. A rule enforced only in the interface can always be reached directly, bypassing it entirely.

5. The Economics of an Exposed API Key

Among all the weaknesses described above, an exposed API key deserves particular attention because its consequences are not only technical but financial, and the financial exposure is often unlimited from the builder's point of view. Many of the AI and cloud services used in modern prototypes are billed by usage, which means a key is, in effect, a line of credit issued in the builder's name.

Figure 3: How One Exposed API Key Becomes Your Bill

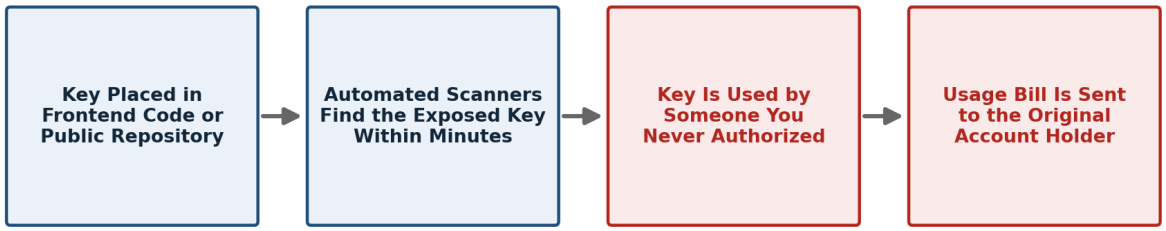


Figure 3. A single exposed key travels quickly from your codebase to an attacker's hands, and the resulting bill lands back on the original account holder.

The practical chain of events tends to follow a predictable pattern:

- A key is placed inside frontend code, or pushed to a public GitHub repository, because the app needs it to work before a deadline.

- Automated scanning tools, run by both researchers and criminals, continuously crawl public repositories for known key formats.
- Once captured, the key can be used at a volume and duration entirely outside the builder's control.
- The usage bill or resource consumption is charged to whoever owns the account the key belongs to, not to whoever misused it.

The uncomfortable fact that student builders need to internalize is this: a hackathon project built to win a weekend prize can, through no additional action by its creator, generate a bill the creator never budgeted for and never agreed to. This is a direct and well documented consequence of how usage based billing and public code hosting interact, and it is entirely preventable through the habits described in section eight of this paper.

6. Hackathon Culture and the Prototype to Production Pipeline

Hackathons are explicitly designed around time pressure, and judges rarely score entries on the robustness of their security posture. This is a reasonable design choice for a weekend event meant to test creativity and execution speed. The risk arises after the event ends, when a project that performed well continues its life well beyond the closing ceremony.

A prototype typically moves through the same informal path:

- It is built quickly over a single weekend to demonstrate an idea.
- It wins attention, a prize, or interest from mentors and peers.
- It becomes a portfolio piece kept running to show future employers.
- It is quietly turned into a small business or adopted informally by a college department.

Figure 4: The Widening Gap Between Growth and Security Effort

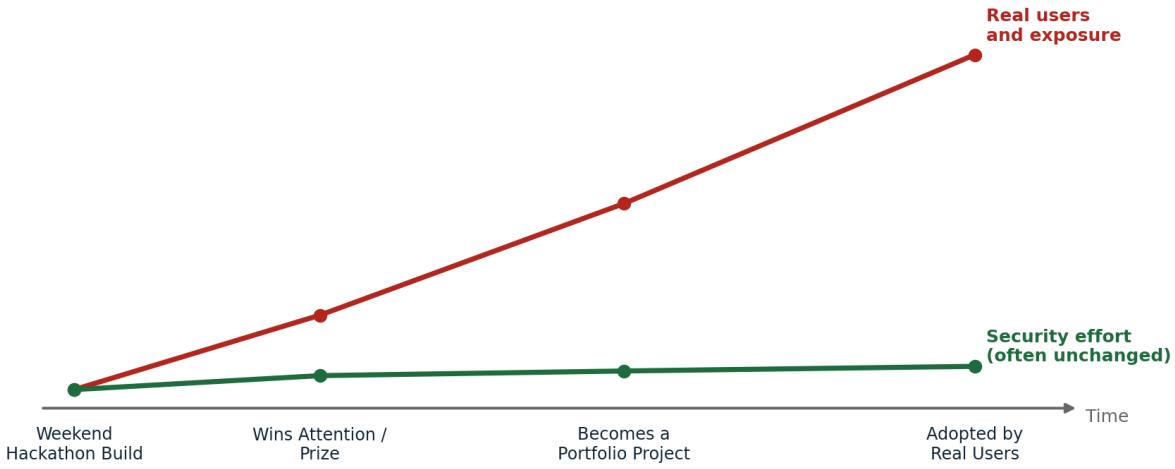


Figure 4. As a project's audience and exposure grow, its security effort usually does not grow with it, widening the gap between how much is at stake and how well it is protected.

At no point in that transition does the underlying code typically receive a security review, because no single moment in that path clearly presents itself as the right time to ask for one. A prototype does not become secure by surviving; it becomes a more attractive target by surviving, because it is now handling real users and often real money while still carrying the security posture of a weekend demo.

7. Public Showcasing and Unintentional Reconnaissance

Sharing a finished project publicly is, in almost every case, a healthy and career positive habit. Recruiters, collaborators, and fellow builders discover talent this way, and there is no reasonable argument for discouraging students from posting their work on platforms such as LinkedIn or

Medium. The point worth raising is narrower: a showcase post often contains far more technical detail than the author realizes, and that detail can double as reconnaissance material for someone looking to attack the very application being celebrated. A typical launch post can unintentionally reveal:

- A live link that points directly at the exact system to target.
- A linked repository that reveals the exact code running behind the product, sometimes including credentials left in old commit history.
- Screenshots of an admin panel that show an attacker exactly what a successful break in would gain them.
- A description of the technology stack that tells an attacker exactly which known weaknesses to try first.

None of this makes public sharing wrong. It makes a brief, deliberate check before posting a sensible habit, in the same way a person checks a photograph for sensitive background details before posting it, without deciding to stop taking photographs altogether.

8. A Practical Checklist for Security Aware Building

The goal of this section is not to turn every student developer into a penetration tester before their next hackathon. The goal is to offer a small set of questions that require no advanced technical background to ask, yet catch a large share of the weaknesses described earlier in this paper. Before a project is deployed or published, a builder should work through the following checklist:

- Search the entire codebase and repository for any API key, password, or token, and assume that anything found there will eventually be found by someone else.
- Test the application as a stranger would, without logging in, to see what remains reachable.
- Try changing an identifying number in a request, such as an order or profile number, to see whether another person's data comes back.
- Confirm that the database has rules restricting who can read or write which records, rather than trusting that the interface alone hides that data from view.
- Confirm that every action reachable from the interface is also checked on the server, since the interface itself protects nothing on its own.
- Look through the repository for configuration files, environment files, and log files that were committed by accident.
- Turn on the spending limits, usage quotas, and permission restrictions that most service providers offer, so a stolen key has a ceiling rather than an open ended cost.
- Treat any exposed credential as compromised the moment it was visible, and rotate it rather than simply deleting it from the latest version of the code.

The mindset underlying this checklist can be summarized in a single shift of attention. A builder's habitual question while working is naturally how do I make this feature work. The additional question a security aware builder learns to ask alongside it is how could this feature be misused. Asking that second question does not require a cybersecurity degree. It requires only the willingness to spend a few minutes looking at one's own creation the way a stranger with bad intentions would.

9. Conclusion

AI assisted development has changed who is able to build software, and that change is worth celebrating. It has not changed the underlying nature of the internet, where anything published publicly can be examined, probed, and, if left unguarded, exploited by someone the builder will never meet. The convenience of generating a working application in an afternoon does not remove the responsibility that has always come with putting software in front of real users. As building continues to become easier for a wider population of students and hobbyists, the

checklist described in this paper, none of which requires deep technical expertise, deserves a permanent place alongside the excitement of shipping something new. The central question for every builder to carry forward is not whether an idea can be built. It is whether what has been built can be trusted to survive contact with someone who wants to break it.

About the author: Abhirup Payne is a cybersecurity student researching application and web security, with a particular interest in the intersection of AI assisted development and secure engineering practice.